Databricks向けJupyter Kernelでデータサイエンティストの開発環境をAI-Readyにする

Mawatari Daiki

2025-12-12

自己紹介

Mawatari Daiki / uma-chan

株式会社GENDA

IT戦略部 データチーム

データエンジニア / MLOpsエンジニア

12/19に本日の登壇内容と同様の記事を投稿予定です！

課題提起

従来のDatabricks Notebook開発の課題

フォーマッタが動かない

マジックコマンドが干渉

Git diff が読めない

.ipynb は JSON 形式
レビューが困難

困っている
データサイエンティスト

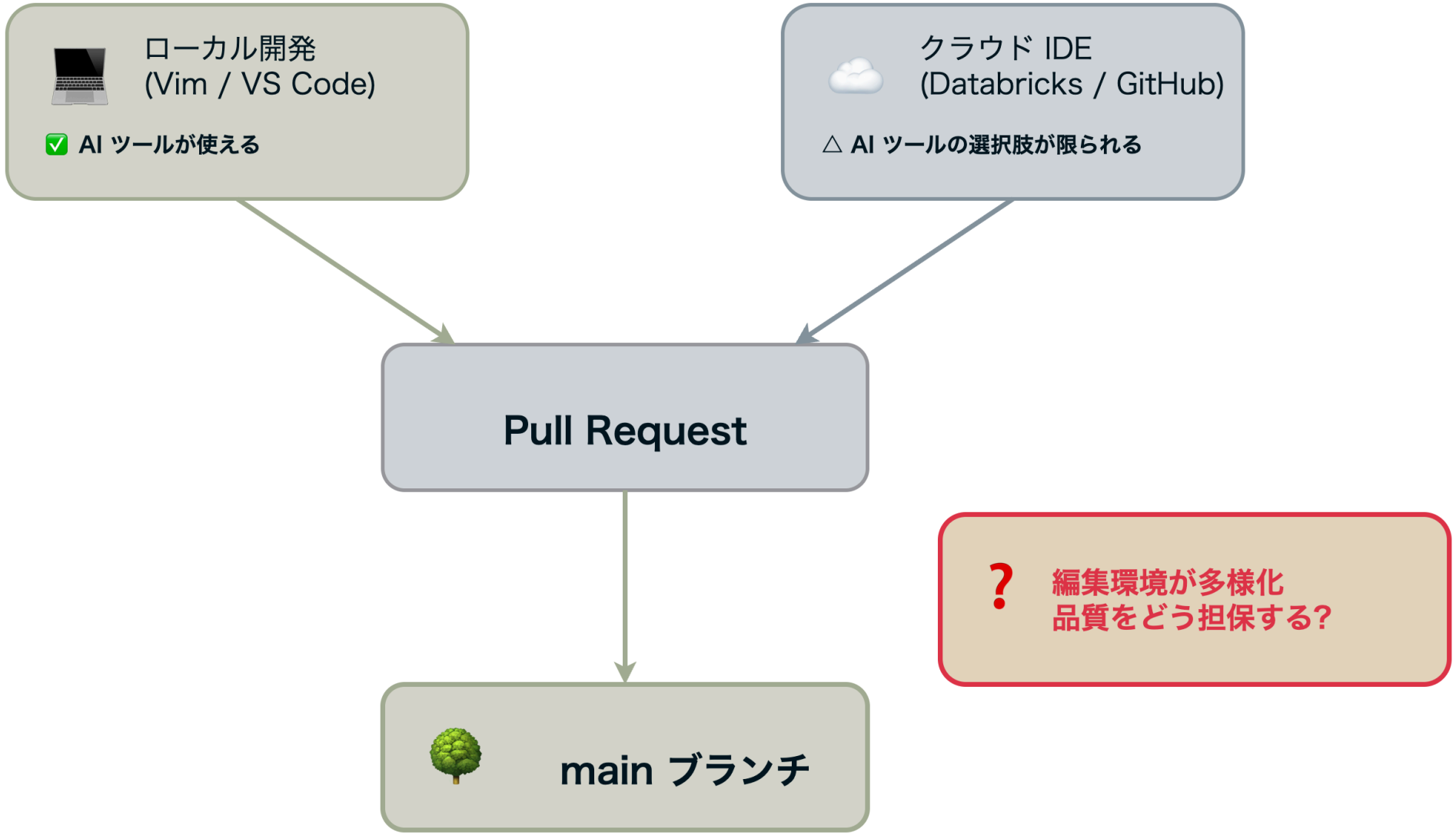
ファイル分割が困難

1つのNotebookに全部書く
巨大になりがち

ローカルテストが困難

pytest が使えない
Databricks 上でしか動かない

AI時代の新たな課題


ローカル開発
(Vim / VS Code)
✅ AI ツールが使える
クラウド IDE
(Databricks / GitHub)
△ AI ツールの選択肢が限られる
Pull Request
main ブランチ
? 編集環境が多様化
品質をどう担保する?

Databricks Notebook開発体験改善策

jupyter-databricks-kernel

Jupyterにおけるカーネルとは

- ノートブックのセルを処理し、結果をフロントエンド (VS Codeなど) に送信するもの

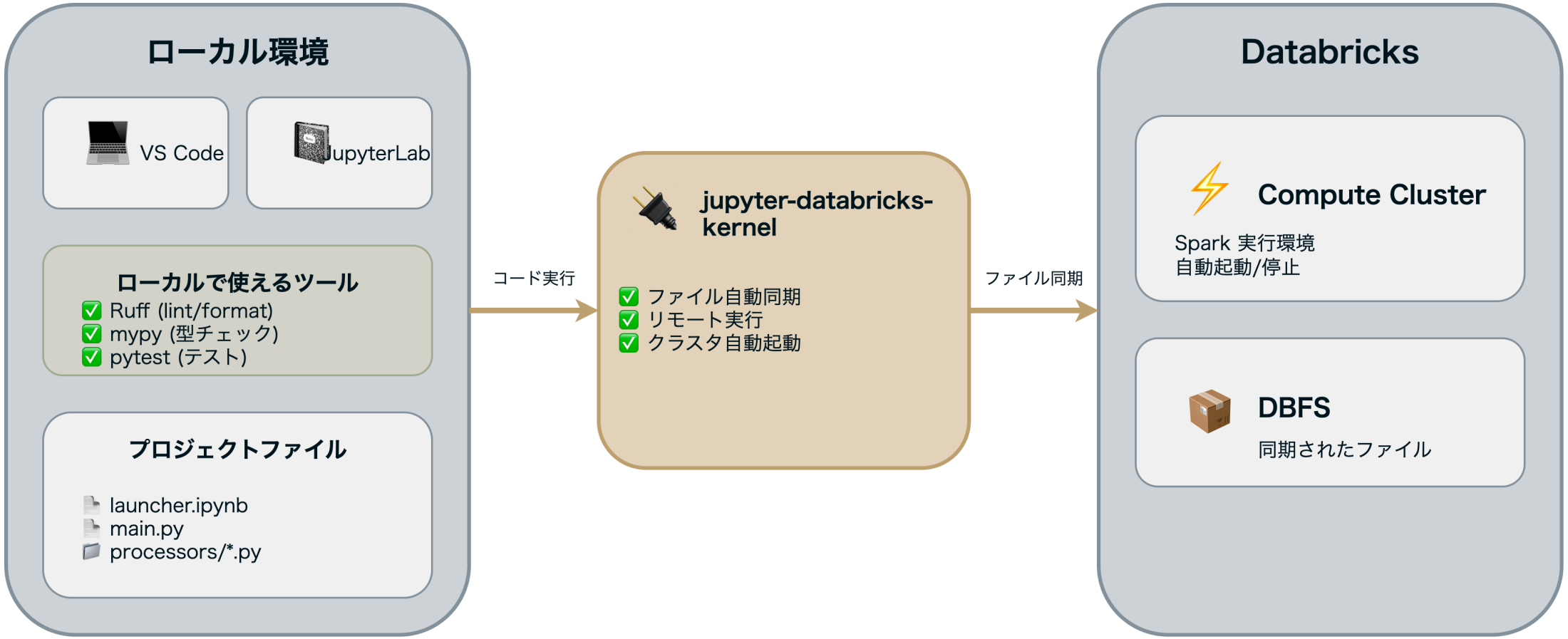
ローカル開発環境を改善すべくDatabricks Computeに接続できるカーネルを作りました！

```
$ jupyter execute notebook.ipynb
```

でノートブック実行できます！

github.com/i9wa4/jupyter-databricks-kernel

jupyter-databricks-kernel



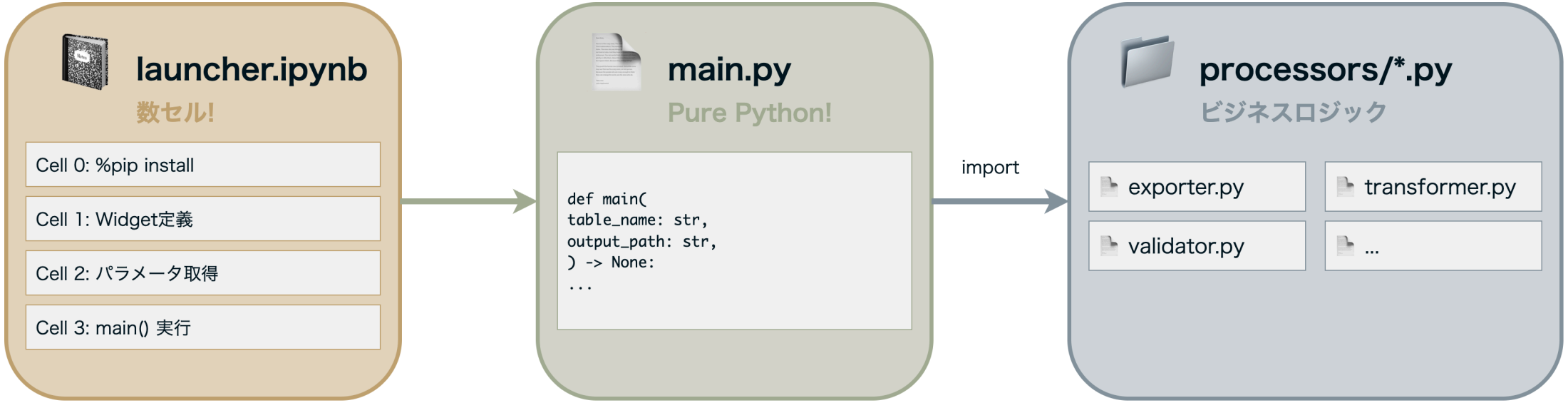
Skinny Notebook Wrapper + Pure Python

以下の構成が使いやすいです

- ノートブックは便利なので使い続ける
- メインロジックは .py ファイルに切り出してノートブックからそれを実行する

詳細は記事に書きました

Skinny Notebook Wrapper + Pure Python



メリット

- ✅ **Ruff/mypy が使える**
  Pure Python だから
  静的解析ツールが動く
- ✅ **Git diff が読みやすい**
  .py ファイルだから
  レビューしやすい
- ✅ **pytest で テスト可能**
  Pure 関数をテスト
  ローカルで TDD
- ✅ **Databricks Job 対応**
  そのまま本番運用
  Widget でパラメータ受け取り

uvによる依存関係管理

uvでPythonパッケージのバージョン管理をしていくと色々と便利になります

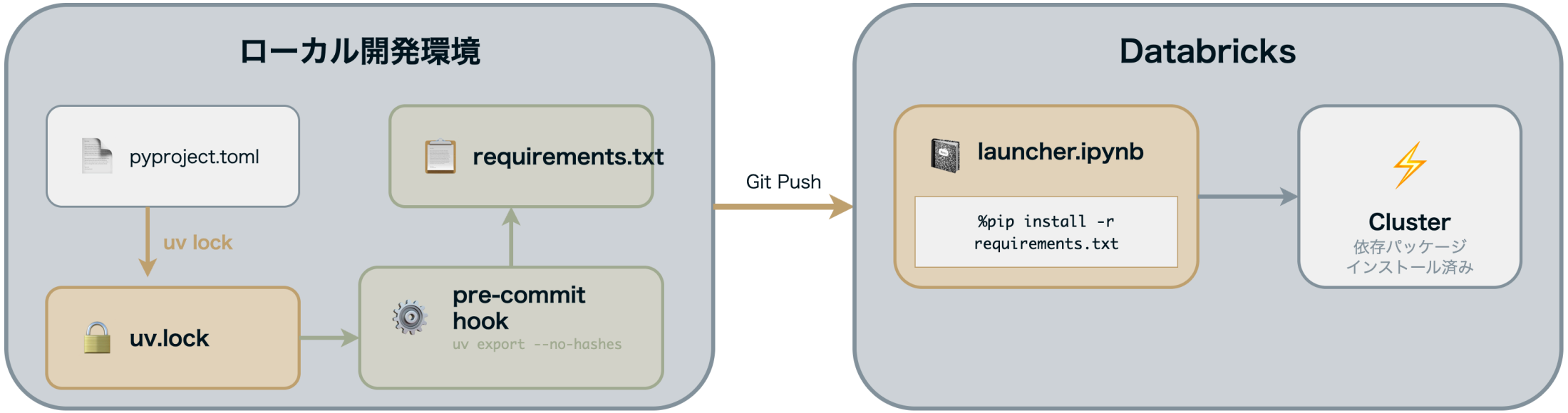
詳細は記事に書きました

Databricks で uv を活用して依存関係を管理する

GENDA
uma-chan

Zenn

uvによる依存関係管理



メリット

- ✅ DBR の Python バージョンに合わせた依存解決
- ✅ pip install の失敗を防止
- ✅ pre-commit で requirements.txt を自動更新
- ✅ Renovate でセキュリティ更新を管理

ガードレール

AI時代にも適応するガードレールを構築していきましょう！

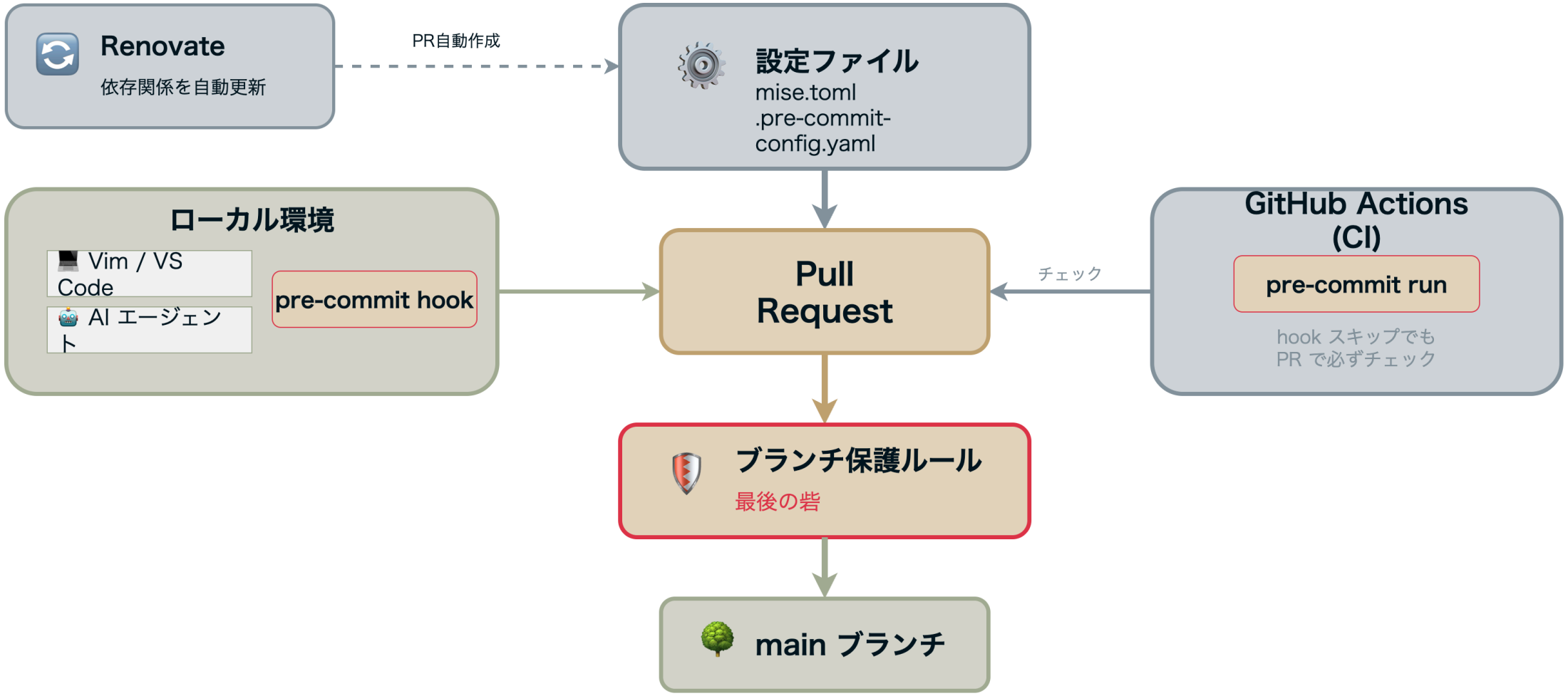
詳細は記事に書きました

mise + pre-commit + Renovate で作る
メンテしやすいガードレール

GENDA
uma-chan

Zenn

ガードレール


Renovate
依存関係を自動更新
PR自動作成
設定ファイル
mise.toml
.pre-commit-
config.yaml
ローカル環境
Vim / VS Code
AI エージェント
pre-commit hook
Pull Request
チェック
GitHub Actions (CI)
pre-commit run
hook スキップでも
PR で必ずチェック
ブランチ保護ルール
最後の砦
main ブランチ

まとめ

紹介した改善策の全体像

1 jupyter-databricks-kernel

- ✅ ローカル Jupyter から Databricks へリモート実行
- ✅ AI ツールが使える
- ✅ ファイル自動同期

2 Skinny Notebook Wrapper

- ✅ 数セルの Notebook
- ✅ Pure Python で開発
- ✅ Ruff/mypy/pytest が動く
- ✅ Git diff が読みやすい

3 uv

- ✅ pyproject.toml で依存管理
- ✅ uv.lock で再現性確保
- ✅ %pip install で Databricks にインストール

4 ガードレール

- ✅ mise + pre-commit + Renovate
- ✅ ローカル/CI で同じチェック
- ✅ AI が作ったコードもチェック
- ✅ ブランチ保護ルール

AI-Ready

最強の Databricks 開発環境

- ローカルで快適に開発
Databricks へリモート実行
- Pure Python で AI と協働
テスト・静的解析が可能
- uv で依存関係管理
Databricks でも安全に実行
- ガードレールで品質担保
誰が書いても同じチェック